Digital VLSI Design

4-bit Array Multiplier from RTL to GDSII

Team Members

Swarnava Pal — Roll No: 2023112008

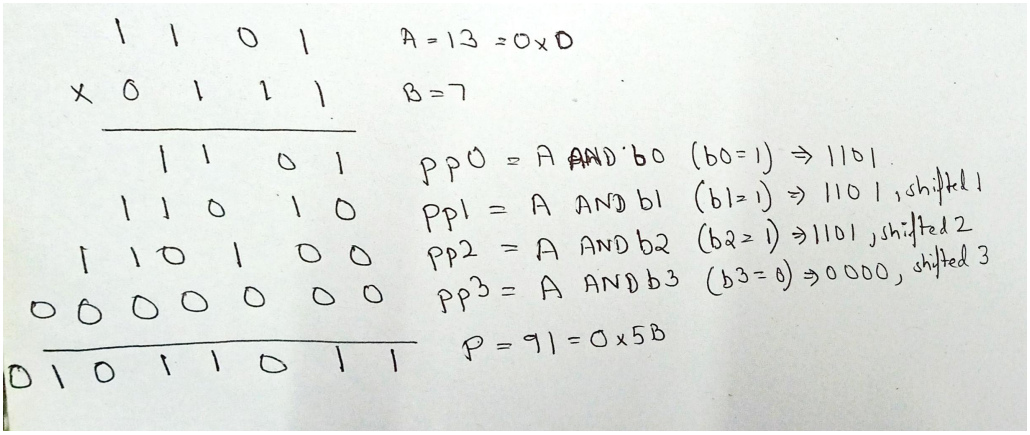
Divesh Kumar — Roll No: 2024102079

1 Part A — RTL Design and Verification

1.1 Hand-Worked Example (Task 5)

Chosen values

- Source of A: Swarnava Pal (2023112008)
- $A = 8 + 5 = 13$
- $B = 7$



1.2 Testbench Results

- **PASS summary:**

```
/foss/designs > iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
/foss/designs > vvp sim
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:65: $finish called at 5146000 (1ps)
```

- **FAIL test from Task 6:**

```
MISMATCH: a=15 b=9 expected=135 (0x87) got=247 (0xf7)
MISMATCH: a=15 b=10 expected=150 (0x96) got=230 (0xe6)
MISMATCH: a=15 b=11 expected=165 (0xa5) got=201 (0xc9)
MISMATCH: a=15 b=12 expected=180 (0xb4) got=196 (0xc4)
MISMATCH: a=15 b=13 expected=195 (0xc3) got=139 (0x8b)
MISMATCH: a=15 b=14 expected=210 (0xd2) got=154 (0x9a)
MISMATCH: a=15 b=15 expected=225 (0xe1) got=181 (0xb5)
FAIL: 256 vectors checked, 95 errors
tb/tb_mult.v:65: $finish called at 5146000 (1ps)
```

1.3 Waveform Screenshot

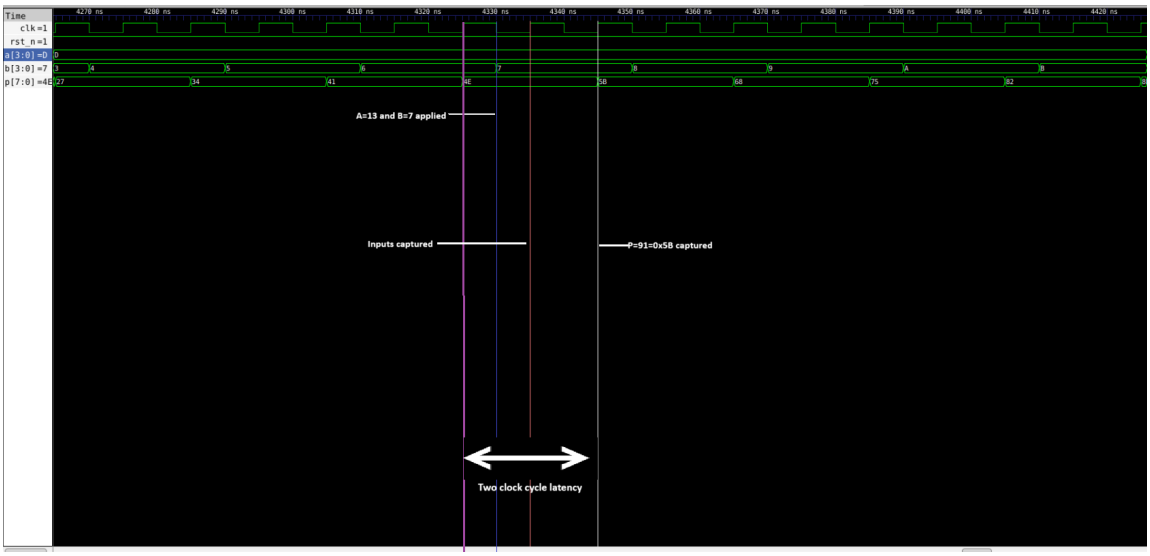


Figure 1: Simulation waveform showing clk, rst_n, a, b, and p and the two-cycle latency marked.

The falling-edge drive is only a testbench convention. It keeps the inputs stable away from the rising edge.

1.4 Block Diagram

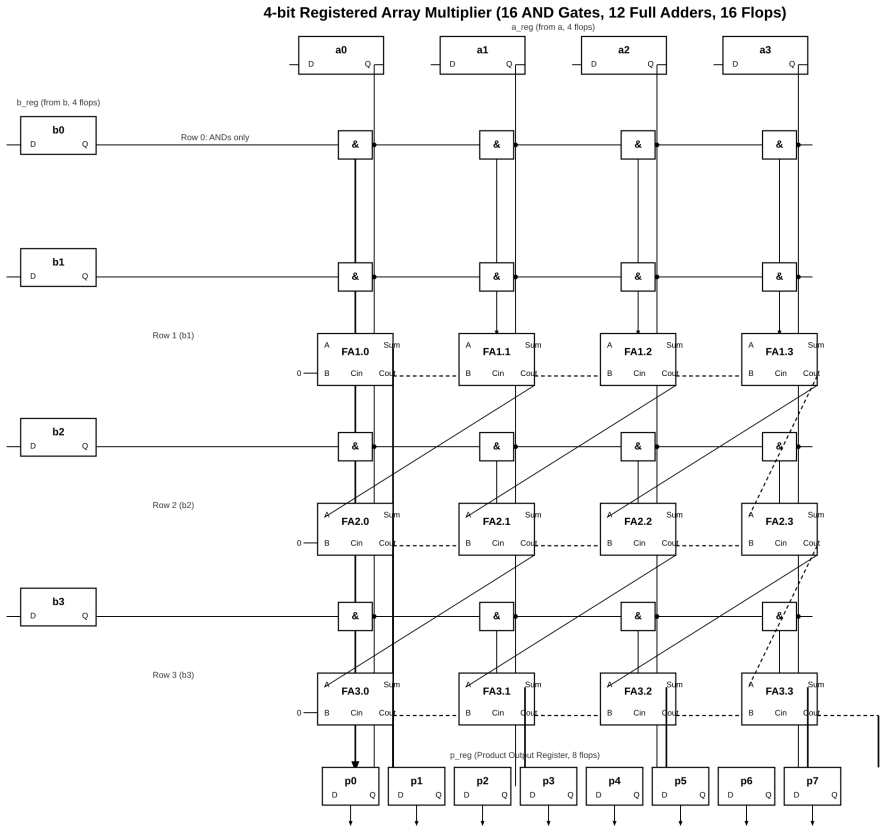


Figure 2: Block diagram of the 4-bit array multiplier, including partial-product generation, adder array, and registers.

2 Part B — Logic Synthesis with Yosys

Table 1: Part B: Synthesis Results

Metric	**Value**
Total number of mapped cells	61
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm^2)	756.976
Flip-flop area (μm^2)	400.384 (16×25.024)
Flip-flop area (% of total)	52.89%
Three most-used cell types and counts	`dfrtp_1`: 16, `nand2_1`: 7, `a2_0`: 4, `maj3_1`: 4
Netlist simulation result	`PASS`

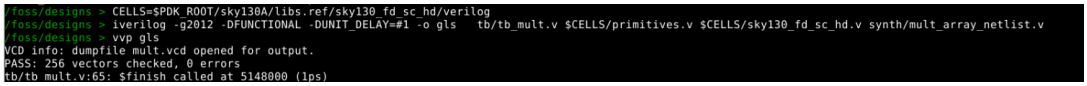


Figure 3: Netlist simulation terminal output

3 Part C — Static Timing Analysis with OpenSTA

$$\begin{aligned} T_{\min} &= T - \mathrm{WNS} \\ &= 10\,\mathrm{ns} - 7.21\,\mathrm{ns} \\ &= 2.79\,\mathrm{ns} \end{aligned}$$

$$\begin{aligned} F_{\max} &= \frac{1000}{T_{\min}} \\ &= \frac{1000}{2.79\,\mathrm{ns}} \\ &\approx 358.42\,\mathrm{MHz} \end{aligned}$$

Table 2: Part C: Static Timing Analysis Results

Metric	**Value**
Worst slack at $T = 10\,\mathrm{ns}$ (ns)	7.21
Minimum clock period, $T_{\min}$ (ns)	$10 - 7.21 = 2.79$
Maximum frequency, $F_{\max}$ (MHz)	$1000/2.79 = 358.4$
Slack at $T = T_{\min}$ (ns)	0.003 (effectively 0)
Critical path start point	`_91_` (flip-flop, `dfrtp_1`, CLK pin)
Critical path end point	`_88_` (flip-flop, `dfrtp_1`, D pin)
Number of cells on critical path	9 (8 logic cells + launching flip-flop)
Latency, $2T_{\min}$ (ns)	$2 \times 2.79 = 5.58$
Throughput (million results/s)	358.4 (1 result/clock)

Table 3: Cells on the critical path, in order

#	**Cell Instance**	**Cell Type / Function**	**Delay (ns)**
1	`_91_`	`dfrtp_1` (clock-to-Q)	0.398
2	`_37_`	`clkinv_1`	0.073
3	`_42_`	`o311ai_0`	0.356
4	`_45_`	`a21oi_1`	0.153
5	`_51_`	`maj3_1`	0.402
6	`_58_`	`xnor3_1`	0.370
7	`_61_`	`maj3_1`	0.368
8	`_63_`	`maj3_1`	0.354
9	`_64_`	`xor3_1`	0.178
Total Arrival Time			**2.652**

The arrival time is 2.652 ns. The setup time is 0.135 ns, so $T_{\min} = 2.652 + 0.135 \approx 2.79$ ns, which matches $10 - 7.21$.

- **Full-Adder Logic (5 cells):**
  - `maj3_1` (3 cells): Computes the carry function (C_{out}).
  - `xor3_1` / `xnor3_1` (2 cells): Computes the sum function (S).
- **Partial-Product & Stage-1 Logic:** `clkinv_1`, `o311ai_0`, and `a21oi_1` form the partial-product AND gates merged with initial-stage logic.
- **Register:** `dfrtp_1` functions as the launching/capturing flip-flop.

4 Part D — Physical Design: RTL to GDSII

Table 4: Part D: Physical Design Results

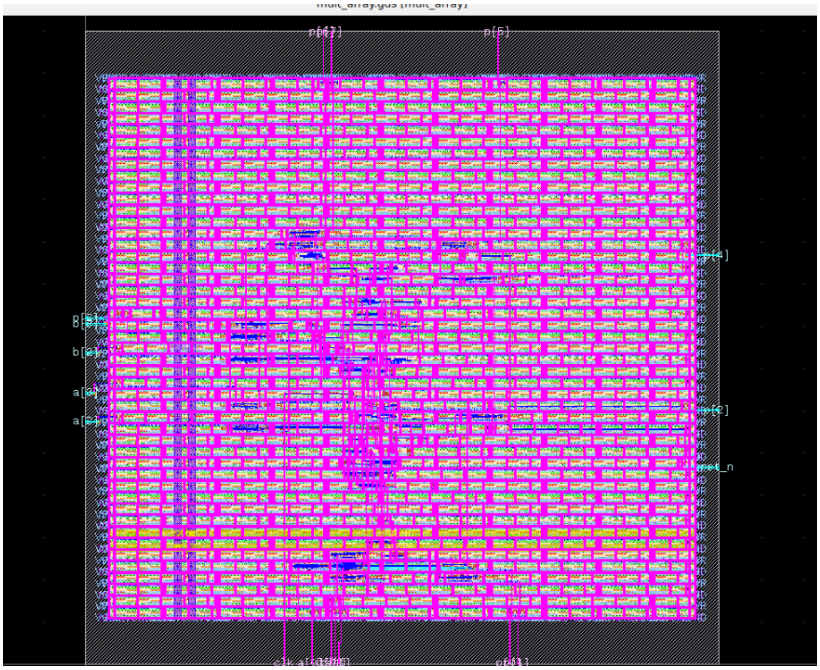
Metric	**Value**
Clock period used (ns)	8.0
Number of physical-design runs	10 $(3.5, 4.0, \ldots, 8.0)$
Core area (μm^2)	17759.5 (die area $22500 = 150 \times 150$)
Mapped cells after synthesis	61
Cells after place and route	331 standard cells (excluding fill) or 1884 (including 1553 fill cells)
Types of cells added	245 tap cells, 1299 decap cells, 254 fill cells, 21 timing-repair buffers, 3 clock buffers
Total wirelength (μm)	2518
Setup slack (ns)	0.362 (worst corner, `max_ss_100C_1v60`)
Post-route maximum frequency (MHz)	$1000/(8.0 - 0.362) = 130.9$
Maximum frequency from Part C (MHz)	358.4
DRC errors	0 (Magic 0, KLayout 0, and the final route DRC went from 2 to 0 in the second iteration)
LVS errors	0

$$
\begin{aligned}
F_{\text{max,post-route}} &= \frac{1000}{\text{CLOCK_PERIOD} - \text{setup slack}} \\
&= \frac{1000}{8.0\,\text{ns} - 0.362\,\text{ns}} \\
&= \frac{1000}{7.638\,\text{ns}} \\
&\approx 130.9\,\text{MHz}
\end{aligned}
$$

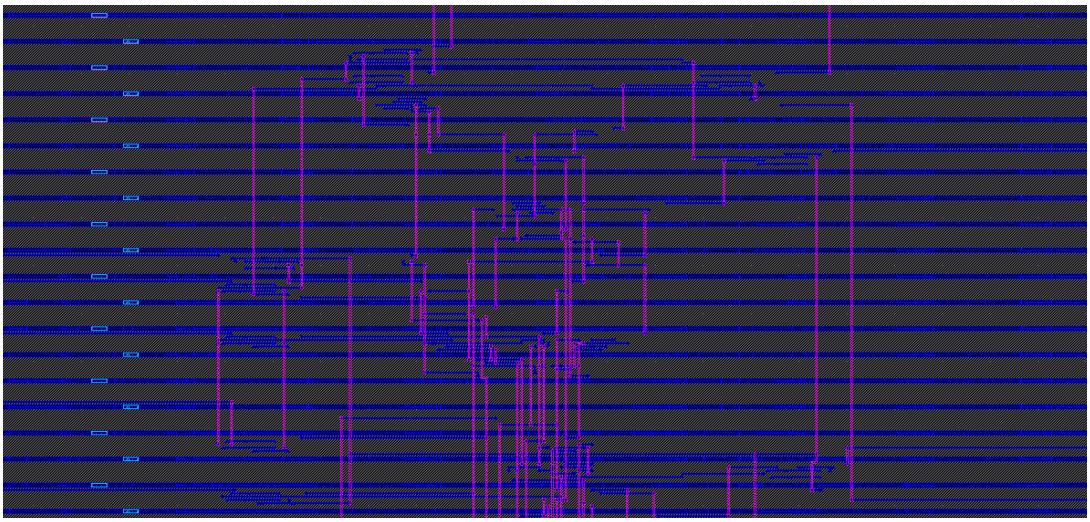
Comparison with Part C:

- **Part C (Pre-layout/Ideal Synthesis $F_{\max}$):** 358.4 MHz ($T_{\min} = 2.79$ ns)
- **Part D (Post-route Actual $F_{\max}$):** 130.9 MHz ($T_{\min} = 7.638$ ns)
- **Impact of Physical Design:** The post-route maximum frequency drops by approximately 63.5% due to parasitic interconnect wire delays, inserted buffer delays, and pessimistic corner timing conditions (`max_ss_100C_1v60`).

Layout:



(a) Full die view



(b) Zoomed view of the core. The hatched horizontal bands are standard cell rows, separated by met1 power rails. met1 (horizontal) and met2 (vertical) carry the signal routing, concentrated around the multiplier logic in the centre.

Figure 4: Final physical layout of mult_array in KLayout.

5 Answers to Report Questions

Question 1: Logic cells after synthesis

Synthesis produced 61 cells in total, of which 16 are flip-flops, so the logic uses $61 - 16 = 45$ cells. This is more than $16 + 12 = 28$ because a full adder is not a single library cell: its sum needs an XOR-type function and its carry needs a majority function, and the 16 partial-product AND gates also have to be implemented. Yosys flattened the design and `abc` then restructured the logic, merging ANDs, inverters and adder parts into mixed cells such as `nand2`, `a21oi`, `a22oi`, `xor3`, `xnor3` and `maj3` (the logic area is $356.6\,\mu\text{m}^2$, an average of about $7.9\,\mu\text{m}^2$ per cell). Constant inputs also let it simplify: the first adder of each row has carry-in 0, and `FA1.3` has a constant 0 input, so those become half adders. As a result the 12 adders and 16 ANDs do not map one-to-one onto 28 cells.

Question 2: Critical path

The critical path runs from input register bit `a_reg[1]` (flip-flop `_91_`) to output register bit `p[6]` (flip-flop `_88_`), whose D input is the sum of the last full adder of row 3 (`FA3.3`). It has 9 cells: the launching flip-flop (clock-to-Q 0.398 ns) followed by 8 logic cells (`clkinv_1`, `o311ai_0`, `a21oi_1`, `maj3_1`, `xnor3_1`, `maj3_1`, `maj3_1`, `xor3_1`). The arrival time is 2.652 ns, and with the setup time of 0.135 ns, $T_{\min} = 2.79$ ns and $F_{\max} = 358.4$ MHz. Five of the cells are full-adder cells (three `maj3` carry cells and the `xnor3`/`xor3` sum cells), fewer than the estimate of about $2N = 8$ full-adder delays. Synthesis merged the partial-product ANDs into the first stage and used fast 3-input XOR and majority cells, so the real depth is shorter than the idealised count. The path follows the ripple structure of the array: a carry chain through the adders towards the most significant sum bit, ending at `p[6]`.

Question 3: Flip-flop area share

The 16 flip-flops occupy $400.384\,\mu\text{m}^2$ out of $756.976\,\mu\text{m}^2$, which is 52.89% of the total cell area. The share is large because a flip-flop is much bigger than a gate: a `dfrtp_1` (with asynchronous reset) is $25.024\,\mu\text{m}^2$, about 6.7 times a `nand2_1` (about $3.75\,\mu\text{m}^2$), and the average logic cell is only about $7.9\,\mu\text{m}^2$. The design is very small, with only 45 logic cells, while it still needs 16 flip-flops to register the $4 + 4$ input bits and the 8 product bits. So the flip-flops are only 26% of the cells (16 of 61) but more than half of the area.

Question 4: Post-route $F_{\max}$ and the added cells

The post-route $F_{\max}$ is $1000/(8.0 - 0.362) = 130.9$ MHz, against 358.4 MHz from Part C. Part C used the typical corner (`tt_025C_1v80`), ideal wires and an ideal clock. The post-route sign-off checks several corners, and the 0.362 ns slack is at the slowest one (`ss_100C_1v60`: slow transistors, 100 °C, 1.60 V), where cells are much slower. It also includes extracted wire resistance and capacitance, the clock tree, and the repair buffers on the paths. At the typical corner the post-route slack is 4.191 ns, which corresponds to about 262 MHz; this is still below 358 MHz because of wire delay and clock skew, and it is only a lower bound because the tool relaxed the design when given a long period. The flow added 245 tap cells (tie the wells and substrate to the supply to prevent latch-up), 1299 decap cells (capacitors between VDD and VSS that reduce supply noise, and which also occupy empty row space), 254 plain fill cells (close the remaining gaps so the cell rows are continuous and density rules are met), 21 timing-repair

buffers (fix slew, fanout and setup problems) and 3 clock buffers (balance the clock arrival at the 16 flip-flops). Only the two kinds of buffers add delay to the signal paths

Question 5: DRC and LVS

A design rule check (DRC) verifies that the layout geometry obeys the foundry's manufacturing rules, such as minimum widths, spacings and enclosures of the layers, so that the shapes can be printed and etched reliably. A layout versus schematic check (LVS) extracts the devices and connections from the layout and compares them with the synthesized netlist, to prove that the layout implements the same circuit with no shorts, opens or missing devices. Both must pass: a layout that is correct electrically but breaks the rules may not be manufacturable or may have low yield, and a layout that obeys the rules but does not match the netlist would be a manufacturable chip that does not do what was designed. The final run has 0 DRC errors (Magic 0, KLayout 0; the router's DRC count went from 2 to 0 in its second iteration), 0 LVS errors and 0 antenna violations.

Question 6: An 8-bit multiplier

With the formulas from Section 2, an 8-bit array needs $N(N-1) = 8 \times 7 = 56$ full adders instead of $4 \times 3 = 12$ (4.7 times as many) and $N^2 = 64$ AND gates instead of 16 (4 times as many); the flip-flops only double, from 16 to $8 + 8 + 16 = 32$. The critical path is about $2N$ full-adder delays, so it grows from about 8 to about 16, which is 2 times longer. Scaling the measured 4-bit logic delay of 2.254 ns by 2 gives 4.51 ns, and adding clock-to-Q (0.398 ns) and setup (0.135 ns) gives $T_{\min} \approx 5.04$ ns, so $F_{\max} \approx 198$ MHz (latency $2 \times T_{\min} \approx 10.1$ ns). The logic area would grow by about 4 to 4.7 times, from 356.6 μm^2 to roughly 1430 to 1670 μm^2, and with the flip-flops at about 800.8 μm^2 the total is about 2230 to 2470 μm^2, so the flip-flop share would fall to roughly 32 to 36%. In summary, the delay grows about linearly with N while the area grows about quadratically. .

6 Contributions

- **Swarnava Pal (Roll No: 2023112008):** Parts A, B, C, and D (RTL design, synthesis, physical design/PnR, timing analysis, and report documentation).
- **Divesh Kumar (Roll No: 2024102079):** Part C (Simulation, verification, and timing analysis and report documentation).

Project Repository:
`https://github.com/swarnavapal/Digital-VLSI-Design`